

Report : Implementation of 3D Table Scene with Walk Around and Orbit Camera Effects in OpenGL .

Objective:

1. To comprehend the principles of the Model, View, and Projection transformations in the current OpenGL rendering pipeline.
2. To use basic forms (cubes) in 3D object construction to create a composite object (a table with legs and a top).
3. To create and implement two different camera control systems: a First-Person Walk camera for human navigation and an Auto-Orbit camera for automatic observation.
4. To incorporate keyboard and mouse input handling in real-time using GLFW.
5. To use GLSL shaders to implement ambient, diffuse, and specular lighting.

Technologies and Tools :

- Programming Language: Python 3.14.0
- Graphics Library: PyOpenGL (OpenGL 3.3 Core Profile)
- Math Library: PyGLM
- Window Management: GLFW
- Numerical Computation: NumPy
- Operating System: Windows

Theory :

1) Modern Programmable Pipeline:

This project uses the modern OpenGL programmable pipeline instead of legacy immediate mode (`glBegin/glEnd`). Rendering is handled using **GLSL Vertex and Fragment Shaders**, with **VBO (Vertex Buffer Object)** and **VAO (Vertex Array Object)** for efficient GPU data processing.

2) 3D Transformations:

- Model Matrix: Transforms objects from local space to world space (position, scale, rotation).
- View Matrix: Defines the camera position and orientation (world → camera space).
- Projection Matrix: Converts 3D coordinates into 2D screen space using perspective projection.

3) Camera Systems:

- Orbit Camera: Rotates around a fixed target (table) using:

- $X = \text{Radius} \times \sin(\text{Time} \times \text{Speed})$
- $Z = \text{Radius} \times \cos(\text{Time} \times \text{Speed})$
- First-Person Camera: Uses Yaw and Pitch angles to control view direction.
Movement is restricted to the XZ plane for walking simulation.

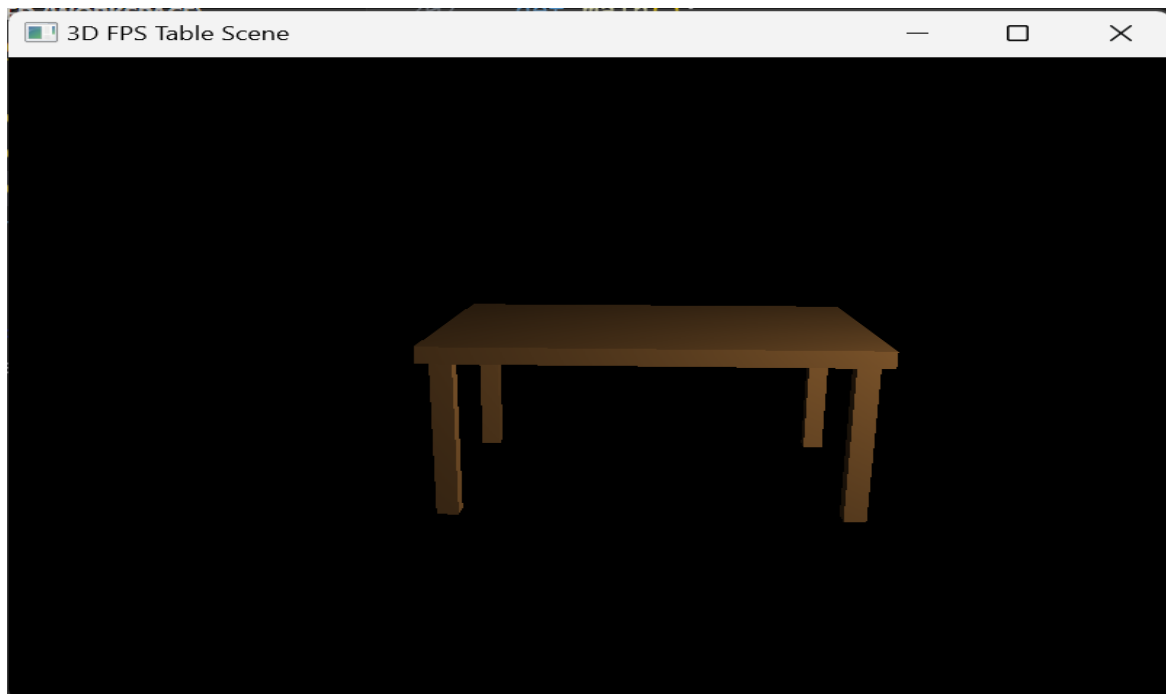
4) Depth Testing: Depth buffering (Z-buffering) is enabled to correctly handle object visibility, ensuring closer objects hide farther ones.

Controls and Functionality:

- C : Toggle between Walk Mode and Auto-Orbit Mode.
- W : Walk camera forward.
- S : Walk camera backward.
- A : Walk camera left (strafe).
- D : Walk camera right (strafe).
- Mouse Movement : Look around (Yaw for horizontal, Pitch for vertical).
- ESC : Exit the program.

Results and Output:

- Visual Output: A 3D table is rendered in the center of the screen with a solid brown/wood-like color. The scene includes directional specular lighting that reacts dynamically as the camera moves.
- Camera Movement: Pressing 'C' smoothly switches the camera from manual free-roam walking to an automated 360-degree rotation around the table.



Console Output:

```
[Running] python -u "e:\Ostad-project\Django\COMPUTER_G\test.py"
Controls:
- WASD + Mouse to Walk Around
- 'C' to toggle Orbit/Loop Mode
- 'ESC' to exit

[Done] exited with code=0 in 10.507 seconds
```

Conclusion:

The successful completion of this project demonstrates several key outcomes in modern 3D graphics development. It effectively shows how complex 3D objects, such as a table, can be constructed by combining simple geometric primitives through scaling and translation. The project also implements two advanced camera systems, including an automated orbit camera and an interactive first-person camera, allowing both dynamic viewing and user-controlled navigation of the scene. Real-time interaction was achieved through proper keyboard and mouse input handling, enabling smooth control over movement and viewing experience. In addition, the core concepts of 3D mathematics—Model, View, and Projection matrices—were successfully applied to manage object positioning, camera perspective, and screen projection. Finally, depth testing using Z-buffering ensured correct rendering order and realistic occlusion, resulting in a visually accurate and well-structured 3D environment.